

MySQL Database Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 2 Months

Level: Beginner

Course page: <https://archerinfotech.in/courses/database-technologies/mysql-training-in-pune>

Course Overview

MySQL remains one of the most-deployed relational databases in Pune — backing most LAMP-stack web applications, the WordPress / WooCommerce ecosystem, the Pune e-commerce / classifieds / SaaS scene, and significant BFSI / IT-services workloads where MySQL has been the institutional default for two decades. Archer Infotech's MySQL training in Pune teaches the database as it is actually used in 2026 — MySQL 8.0+ as the production default (8.4 LTS released April 2024 is the current LTS; 9.x is the innovation release stream), modern SQL features (window functions, CTEs, JSON functions, lateral derived tables), database design and normalisation, transactions and isolation, indexing and query optimisation with EXPLAIN, stored procedures / functions / triggers (where they earn their place), plus the production tail (replication, backup, MySQL Workbench / DBeaver, security baseline). Classroom in Kothrud, online live, and weekend batches available.

Why Learn MySQL in 2026

MySQL is among the most-deployed databases in Indian production workloads — Indeed Pune lists 600+ active openings that explicitly require MySQL, plus another ~1,000 backend / full-stack roles where MySQL is listed alongside Postgres / Mongo as 'one of'. The biggest employers running MySQL at scale are the WordPress / WooCommerce hosting / agency scene (every Pune digital agency builds on MySQL), e-commerce (Flipkart Pune teams, smaller marketplaces), BFSI legacy systems, plus the LAMP-stack startup ecosystem. MySQL also powers Amazon RDS / Aurora MySQL, Azure Database for MySQL, and Cloud SQL MySQL — managed cloud variants that several Pune teams ship on.

What changed in 2026: MySQL 8.0 (released 2018, the long-running mainline) has been refreshed continuously — InnoDB Cluster for HA, JSON functions at parity with Postgres in many cases, window functions, CTEs (the 8.0 features that closed the gap with Postgres). MySQL 8.4 LTS (April 2024) is the current LTS for stable production. MySQL 9.x is the innovation stream with vector-search support and other modern features. Cloud-managed MySQL (RDS / Aurora MySQL / Azure MySQL Flexible / Cloud SQL MySQL) has matured significantly.

What this means for hiring: 2026 Pune MySQL JDs expect SQL fluency at the window-functions / CTE level, indexing and EXPLAIN-plan literacy, transactions and isolation, plus basic replication understanding. Senior DBA roles add InnoDB Cluster, performance tuning at depth, and managed-cloud MySQL operations. Archer Infotech's curriculum is rebuilt around exactly these expectations — modern SQL, performance-aware, cloud-ready.

- 600+ active Pune openings explicitly require MySQL (May 2026)
- Plus ~1,000 backend / full-stack roles list it alongside Postgres / Mongo
- MySQL 8.0+ — window functions, CTEs, JSON parity with Postgres
- MySQL 8.4 LTS — current production LTS
- Strong Pune e-commerce / digital-agency / BFSI / LAMP startup hiring

Who This Course Is For

This course fits you if

- Engineering, BCS, MCA, or BSc-CS student wanting solid relational database foundations
- Working backend developer wanting MySQL depth (the database your team probably runs)
- Working Data Analyst or BA wanting SQL-on-MySQL for daily analytics work
- PHP / WordPress / LAMP developer wanting to deepen SQL fluency
- Career restarter targeting Database Administrator / Backend Developer roles
- Working Oracle / PostgreSQL DBA wanting to add MySQL for the wider Pune database market

This course is not the right fit if

- If you want enterprise-grade DBA depth for BFSI Pune Capital Markets — pick our Oracle Database course instead (Pune BFSI runs more Oracle than MySQL at the high end)
- If you specifically want NoSQL / document-database depth — pick our MongoDB course
- If you want modern Pune product-engineering Postgres depth — pick our PostgreSQL course
- If you cannot put in 6–8 hours per week of practice outside class
- If you have 3+ years of production MySQL DBA experience — talk to us about advanced specialisations

Detailed Curriculum

Module 1: Relational Foundations & the MySQL Server (Week 1)

What a relational database actually is, established before any query is written. Tables, rows, columns and the relational model as a way of removing duplication rather than a filing metaphor; where a database earns its place over a spreadsheet or a file; and how a client, the MySQL server and a storage engine divide the work between them.

MySQL 8.4 LTS is installed on Linux, macOS, Windows and Docker, and the three interfaces used for the rest of the course are set up together — the `mysql` command-line client, MySQL Workbench, and DBeaver — so a student can move between them without losing their place. The week closes on server configuration basics: `my.cnf`, character sets, and why `utf8mb4` is the only correct choice in 2026.

- The relational model — tables, rows, columns, keys
- When a database beats a spreadsheet or a flat file
- Client, server and storage engine — who does what
- MySQL 8.4 LTS install on Linux, macOS, Windows, Docker
- The `mysql` CLI, MySQL Workbench and DBeaver
- Server configuration and `my.cnf` basics
- Character sets and collations — `utf8mb4` and why
- InnoDB vs MyISAM and the end of that debate

Module 2: Core SQL — SELECT, DML and DDL (Week 1)

The statements that make up most working days. SELECT with every clause in the order the server evaluates them — FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT — taught in evaluation order rather than in writing order, because that is what explains why an alias works in ORDER BY and fails in WHERE.

Aggregate functions, DISTINCT, CASE expressions and NULL handling follow, with NULL given proper time: three-valued logic quietly produces wrong results in reports, and COALESCE, IFNULL and IS NULL are the fix. INSERT, UPDATE, DELETE and the DDL statements close the week, along with the habit of running a SELECT before every UPDATE.

- SELECT clauses in logical evaluation order
- WHERE, comparison, LIKE, IN, BETWEEN and regular expressions
- GROUP BY, HAVING and the aggregate functions
- DISTINCT, aliases and expression columns
- NULL, three-valued logic, COALESCE and IFNULL
- CASE expressions and conditional aggregation
- INSERT, UPDATE, DELETE and safe-update mode
- CREATE, ALTER and DROP TABLE
- Reading and fixing common SQL error messages

Module 3: Data Types, Constraints & Table Design (Week 2)

Choosing the right column type, which is a decision that is expensive to reverse and is asked about in almost every database interview. Integer widths and signedness, DECIMAL versus FLOAT for money (and why the second one loses rupees), CHAR versus VARCHAR versus TEXT, and the date and time family including TIMESTAMP versus DATETIME and how each behaves across time zones.

Constraints are then treated as the database enforcing what the application forgot: primary keys, foreign keys with the full set of ON DELETE and ON UPDATE actions, UNIQUE, CHECK (genuinely enforced since 8.0), NOT NULL and defaults. Generated columns, ENUM and SET, and the auto-increment behaviour that surprises people after a rollback close the module.

- Integer types, widths, signedness and storage cost
- DECIMAL vs FLOAT vs DOUBLE — money never floats
- CHAR, VARCHAR, TEXT and BLOB trade-offs
- DATE, DATETIME, TIMESTAMP and time-zone behaviour
- Primary keys, natural vs surrogate
- Foreign keys and ON DELETE / ON UPDATE actions
- UNIQUE, CHECK, NOT NULL and DEFAULT
- Generated columns, ENUM and SET
- AUTO_INCREMENT behaviour and gaps

Module 4: Joins, Subqueries & Set Operations (Week 2)

Combining tables, which is where SQL stops being a reporting language and starts being a modelling one. INNER, LEFT, RIGHT, CROSS and self joins are drawn before they are written, and the two classic mistakes are made deliberately in class: the accidental cartesian product, and the LEFT JOIN silently turned into an inner join by a WHERE clause on the right-hand table.

Subqueries follow in all three positions — scalar, IN list and derived table — with the correlated and uncorrelated distinction made concrete through execution cost. Anti-joins with NOT EXISTS, semi-joins, UNION and UNION ALL, and the multi-table UPDATE and DELETE forms complete the module.

- INNER, LEFT, RIGHT, CROSS and self joins
- Join diagrams before join syntax
- The accidental cartesian product and how to spot it
- How WHERE turns a LEFT JOIN into an INNER JOIN
- Scalar, IN-list and derived-table subqueries
- Correlated vs uncorrelated and what each costs
- Anti-joins and semi-joins with EXISTS / NOT EXISTS
- UNION vs UNION ALL and emulating INTERSECT / EXCEPT
- Multi-table UPDATE and DELETE

Module 5: Modern SQL — CTEs, Window Functions & JSON (Week 3)

The 8.0 features that closed the gap with PostgreSQL and that most working developers still do not use. Common table expressions turn a nested three-level subquery into something a colleague can read, and recursive CTEs walk org charts, category trees and bill-of-materials structures without application-side loops.

Window functions are taught as the tool for the problems that used to require a self-join: running totals, per-group ranking, month-on-month change with LAG and LEAD, moving averages over a frame, and top-N-per-group. The module ends on native JSON — the document type, the path operators, JSON_TABLE for reading JSON as rows, and an honest rule for when a JSON column is the right design and when it is a normalised table avoiding its responsibilities.

- Common table expressions and readable query structure
- Recursive CTEs for trees and hierarchies
- Window functions — PARTITION BY, ORDER BY, frames
- ROW_NUMBER, RANK, DENSE_RANK and top-N-per-group
- LAG, LEAD and period-over-period comparison

- Running totals and moving averages
- The JSON type and path operators
- JSON_TABLE, JSON_EXTRACT and generated-column indexing
- When a JSON column is right and when it is avoidance

Module 6: Normalisation, Data Modelling & Schema Migrations (Week 3)

Designing a schema that survives its second year. Entity-relationship modelling from a written requirement, then normal forms taught as the reasoning behind them rather than as definitions to recite — first, second, third and Boyce-Codd, each introduced by the update anomaly it removes. Denormalisation follows as a deliberate, measured performance decision with a stated cost.

The second half is schema evolution, which is where production systems are actually damaged. Migration tooling (Flyway, Liquibase, and the migration layers inside Django, Rails, Laravel and Spring), expand-and-contract as the pattern that lets a column be renamed without downtime, online DDL and ALGORITHM=INSTANT in 8.0, and the rule that a column is never dropped in the same release that stops using it.

- ER modelling from a written requirement
- 1NF, 2NF, 3NF and BCNF — the anomaly each removes
- Denormalisation as a measured decision
- Modelling one-to-many, many-to-many and hierarchies
- Soft deletes, audit columns and history tables
- Migrations with Flyway, Liquibase and framework tooling
- Expand-and-contract for zero-downtime changes
- Online DDL and ALGORITHM=INSTANT in 8.0
- Seed data, environments and reproducible schemas

Module 7: InnoDB Internals & Index Design (Week 4)

Why the same query is fast on one table and slow on another. InnoDB's clustered index means rows are physically stored in primary-key order, and that single fact explains the cost of a random UUID primary key, the size of every secondary index, and what a secondary-index lookup actually does.

Index design is then built up deliberately: B-tree structure, the leftmost-prefix rule that decides whether a composite index is used at all, column ordering driven by selectivity and by the query's WHERE and ORDER BY shape, covering indexes that avoid the row lookup entirely, and prefix indexes on long strings. Full-text and multi-valued indexes, the cost of over-indexing on write throughput, and finding unused indexes close the module.

- The clustered index and primary-key physical ordering
- Why random UUID primary keys hurt, and what to use instead
- Secondary indexes and the row lookup they imply
- B-tree structure and the leftmost-prefix rule
- Composite index column order and selectivity
- Covering indexes and index-only scans
- Prefix indexes on long string columns
- Full-text and multi-valued indexes
- The write cost of over-indexing, and finding unused indexes

Module 8: EXPLAIN, Query Optimisation & Slow-Query Analysis (Week 5)

Reading what the optimiser decided instead of guessing. EXPLAIN and EXPLAIN ANALYZE line by line — the access-type ladder from const and eq_ref through ref and range down to a full ALL scan, the key and rows columns, and the Extra field where Using filesort and Using temporary announce the real problem.

The work then becomes diagnostic. The slow query log and performance_schema are used to find which query to fix first, on a dataset of over ten million rows so that the difference is measurable rather than theoretical. Common rewrites are practised — sargable predicates, avoiding functions on indexed columns, keyset pagination instead of a large OFFSET, and fixing the ORM N+1 pattern that produces a thousand queries where one would do.

- EXPLAIN and EXPLAIN ANALYZE, column by column
- The access-type ladder — const, eq_ref, ref, range, index, ALL
- Using filesort and Using temporary — what they cost
- The slow query log and performance_schema
- Sargable predicates and functions on indexed columns
- Keyset pagination instead of large OFFSET
- Diagnosing and fixing ORM N+1 query patterns
- Optimizer hints and when not to use them
- Benchmarking on a 10M+ row dataset

Module 9: Transactions, Isolation Levels & Locking (Week 5)

Correctness under concurrency, the topic that separates a developer who has read about ACID from one who has debugged a deadlock at 2 a.m. Transactions, commit and rollback, savepoints, and the four isolation levels demonstrated against each other in two live sessions so that dirty reads, non-repeatable reads and phantoms are observed rather than described.

InnoDB's locking model is then opened up: row locks, gap and next-key locks, `SELECT ... FOR UPDATE` and `FOR SHARE`, and why `REPEATABLE READ` — MySQL's default — behaves differently from PostgreSQL's. Deadlocks are produced deliberately, read from `SHOW ENGINE INNODB STATUS`, and then designed out through consistent lock ordering and short transactions.

- Transactions, COMMIT, ROLLBACK and savepoints
- ACID properties as observable behaviour
- The four isolation levels, demonstrated side by side
- Dirty reads, non-repeatable reads and phantoms
- InnoDB row, gap and next-key locks
- `SELECT ... FOR UPDATE` and `FOR SHARE`
- Reading `SHOW ENGINE INNODB STATUS` after a deadlock
- Designing deadlocks out with lock ordering
- Optimistic vs pessimistic concurrency in application code

Module 10: Stored Programs — Procedures, Functions, Triggers & Events (Week 6)

Server-side logic, taught with an honest account of when it is right. Stored procedure and function syntax, parameters, variables, control flow, cursors, error handlers and SIGNAL, then triggers for BEFORE and AFTER events at row level, and the scheduled event for recurring work.

The judgement matters as much as the syntax. Legacy Pune systems in BFSI and ERP carry substantial procedural logic and a developer joining one must be able to read and modify it; new systems generally put business rules in application code, where they can be unit-tested and reviewed. Both positions are argued, with the specific cases where a trigger or procedure genuinely wins — audit capture, data integrity that must hold across every client, and set-based bulk work.

- Stored procedure and function syntax and parameters
- Variables, control flow and cursors
- Error handlers, SIGNAL and RESIGNAL
- Triggers — BEFORE and AFTER, INSERT, UPDATE, DELETE
- Scheduled events for recurring work
- Views and updatable views
- Reading and modifying legacy procedural code
- When application code is the better home for business rules
- Debugging, versioning and testing stored programs

Module 11: Application Integration & the Developer Toolchain (Week 6)

Connecting MySQL to the code that uses it. Drivers and connection handling across the stacks Pune teams run — JDBC with Spring Boot and Hibernate, Python with SQLAlchemy and Django, Node.js with Prisma and mysql2, PHP with PDO — and connection pooling explained through what happens when the pool is exhausted under load.

Parameterised statements are treated as non-negotiable, with SQL injection demonstrated against a deliberately vulnerable endpoint and then closed. Credentials move to environment variables and a secret store; transactions are managed at the service boundary rather than per statement; and the whole of this course's work is kept in Git with branches, pull requests, a README and a documented schema, because the portfolio is what a hiring panel actually opens.

- JDBC, SQLAlchemy, Django ORM, Prisma, mysql2 and PDO
- Connection pooling, pool exhaustion and timeouts
- Parameterised statements and SQL injection, demonstrated
- Credentials in environment variables and secret stores
- Transaction boundaries in application services
- ORM lazy loading, N+1 and when to drop to raw SQL
- Git, branches, pull requests and code review
- Documenting a schema — ER diagram, README, data dictionary
- Seeding, fixtures and test databases

Module 12: Users, Privileges, Security & Compliance (Week 7)

Locking the server down to what each caller genuinely needs. Account creation and the host part of a MySQL user that catches people out, the privilege hierarchy from global down to column level, roles in 8.0, and least privilege applied to a real application account rather than the root connection string that so many projects ship with.

Transport and storage security follow: TLS for client connections, `caching_sha2_password` as the 8.0 default, encryption at rest for tablespaces, and column-level protection for personal data. The module closes on the compliance shape Indian teams now work inside — India's DPDP Act and GDPR where clients are European — covering PII inventory, retention, the audit log, and deletion requests that a schema must be able to honour.

- Users, the host component and account management
- The privilege hierarchy and GRANT / REVOKE
- Roles in MySQL 8.0 and least privilege in practice
- Why the application must never connect as root
- TLS for client connections and `caching_sha2_password`
- Encryption at rest and column-level protection for PII
- Audit logging and who changed what
- DPDP and GDPR obligations a schema must support
- A hardening checklist for a new server

Module 13: Backup, Recovery & Point-in-Time Restore (Week 7)

A backup nobody has restored is not a backup. Logical backups with `mysqldump` and `mysqlpump`, physical backups with Percona XtraBackup while the server stays online, and the binary log as the record that makes point-in-time recovery possible.

Every student then performs a real restore under exercise conditions: drop a table, recover the database to the second before the mistake, and verify the row counts. Backup strategy is designed rather than assumed — full and incremental schedules, retention, offsite copies, encryption, and the recovery point and recovery time objectives that decide all of it. The module closes on the failure modes that ruin restores: a backup that was never tested, a binary log that had been purged, and a dump taken without `--single-transaction`.

- `mysqldump`, `mysqlpump` and consistent logical backups
- Percona XtraBackup for hot physical backups
- The binary log and point-in-time recovery
- A full restore exercise, verified by row count
- Full and incremental schedules and retention
- RPO and RTO as the drivers of backup design
- Offsite copies and encrypted backups
- Restoring a single table from a full dump
- The failure modes that break restores

Module 14: Replication, High Availability & Managed Cloud MySQL (Week 8)

Running MySQL on more than one machine. Asynchronous source-replica replication built from scratch in class, GTID-based positioning that makes failover clean, semi-synchronous replication and what it does and does not guarantee, replica lag and how it produces stale reads in an application that assumed otherwise, and read-write splitting.

Group Replication and InnoDB Cluster provide the automated-failover story, with ProxySQL or MySQL Router in front. The module ends on managed cloud MySQL, which is how most new Pune workloads are deployed — Amazon RDS and Aurora MySQL, Azure Database for MySQL Flexible Server, and Cloud SQL — comparing what the provider takes over, what it costs, and which of the skills from this course still matter when it does.

- Asynchronous source-replica replication, built in class
- GTID-based replication and clean failover
- Semi-synchronous replication and its guarantees
- Replica lag, stale reads and read-write splitting
- Group Replication and InnoDB Cluster
- ProxySQL and MySQL Router
- Amazon RDS and Aurora MySQL
- Azure Database for MySQL and Cloud SQL
- What managed hosting takes over, and what it does not

Module 15: Capstone Project & Interview Preparation (Week 9 + placement prep)

The capstone runs the whole course through one system: a schema modelled from a written brief, loaded with millions of rows, queried through reports built on CTEs and window functions, tuned against measured EXPLAIN output, wrapped in transactions that behave under concurrency, secured with least-privilege accounts, backed by a tested restore procedure, and documented well enough that a stranger can run it.

Interview preparation targets what Pune panels actually ask a database candidate: write a query on a whiteboard, explain why an index is not being used, describe an isolation level and the anomaly it prevents, normalise a badly designed table, and talk through a slow query you personally fixed. Resume, LinkedIn and GitHub are rewritten around the capstone, and two mock interviews are run and reviewed.

- Capstone schema, data load and report queries
- A measured before-and-after tuning exercise
- Concurrency and transaction behaviour under load
- Least-privilege accounts and a tested restore
- README, ER diagram and data dictionary
- Live query-writing practice under interview conditions
- Explaining index choice, isolation levels and normalisation
- Resume, LinkedIn and GitHub rewritten around the capstone
- Two mock interviews with recorded feedback

Module 16: AI-Assisted Development Workflow (Final Week)

The skill every 2026 hiring panel now probes for — building real work with AI in the loop, responsibly. Learn to drive AI assistants (GitHub Copilot, Claude, Cursor, and IDE-native AI) to scaffold and accelerate the tools and stack this course covers, generate tests, explain and refactor unfamiliar code, and cut the boilerplate — while keeping you firmly in control of every decision. Heavy focus on guardrails: reviewing each AI suggestion, spotting hallucinated APIs or wrong answers, and handling licensing and data-privacy concerns. Close with a mini-project that takes a deliverable end-to-end using an AI-assisted workflow, then fold the same tooling into version control and everyday team practice.

- AI assistants — GitHub Copilot, Claude, Cursor, IDE-native AI
- Effective prompting for this course's stack — scaffolding, boilerplate, config
- AI-assisted test generation and coverage
- Explaining, refactoring, and modernising unfamiliar code with AI
- AI-driven review, error detection, and quality checks
- Generating and maintaining documentation with AI

- AI debugging — interpreting errors, logs, and failing output
- Guardrails — reviewing output, avoiding hallucinations, licensing & data privacy
- Team workflow — AI in the editor, in reviews, and in delivery pipelines
- Mini-project — a deliverable built end-to-end with an AI-assisted workflow

Capstone Projects

E-Commerce Schema with Performance-Tuned Queries

Design and implement a complete e-commerce schema (users, products, categories, orders, order_items, payments, addresses, reviews) with proper normalisation, indexes, foreign-key cascades, and JSON columns where they earn their place. Load 10M+ rows of synthetic data, write 20 analyst queries with window functions / CTEs, optimise each via EXPLAIN, and document before / after performance numbers. Outcome: a public GitHub repository with the schema, sample data, and optimisation report.

- Technology: MySQL 8.4 LTS
- Technology: Modern SQL — window functions, CTEs, JSON
- Technology: EXPLAIN-plan optimisation
- Technology: 10M+ row dataset
- Technology: GitHub repo with documentation

Replicated Production-Style MySQL Setup with Backup

A production-style MySQL setup — primary + replica via GTID-based replication, automated mysqldump backups via cron, point-in-time recovery testing (binary log replay), plus a small script that simulates failover. Demonstrates the operational discipline Pune DBA panels test for.

- Technology: MySQL 8.4 LTS
- Technology: GTID replication
- Technology: mysqldump + binary log
- Technology: Failover scripting

Career Outcomes

MySQL fluency is foundational to most Pune backend / full-stack / data-analyst roles — Indeed Pune lists 600+ active openings explicitly requiring MySQL, plus ~1,000 backend / full-stack roles where MySQL is listed alongside other databases. The biggest employers are the Pune e-commerce / digital-agency scene, BFSI legacy systems, plus most LAMP-stack startups. Database Administrator titles at the senior level pay well — Senior DBAs in Pune earn Rs. 15–25 lakh.

What pulls a MySQL developer / DBA above the median band: depth on EXPLAIN plans and query optimisation, demonstrable schema-design experience with at least one 10M+ row dataset, replication / backup / recovery experience, plus security-baseline awareness. Our capstone projects are designed exactly around these signals.

Senior MySQL DBA bands at the top end are reported as national figures (Pune-specific Indeed pages do not exist for these specific titles); Pune trends within $\pm 10\%$ of these figures.

Roles after this course

- MySQL Developer
- Junior Database Administrator
- Backend Developer (with MySQL depth)
- Data Analyst (SQL-heavy)
- Junior DBA

Indicative salary bands

- MySQL Developer / DBA (Pune) - Rs. 6,12,000 per year average (source: Indeed Pune (MySQL Developer))
- Junior Database Administrator (Pune entry, <2 years) - Rs. 3,50,000 – Rs. 6,00,000 per year (source: AmbitionBox Pune Database Administrator)

- Mid-level DBA (Pune, 3–5 years) - Rs. 9,00,000 – Rs. 15,00,000 per year (source: Glassdoor Pune Database Administrator)
- Senior DBA (Pune, 5–8 years) - Rs. 15,00,000 – Rs. 25,00,000 per year (source: Glassdoor Pune Senior DBA)

Companies hiring for these skills

- Persistent Systems
- TCS
- Infosys
- Cognizant
- Capgemini
- Wipro
- Atos / Eviden
- Mphasis
- Synechron
- Pune-based digital agencies (LAMP / WordPress)
- Pune-based e-commerce companies
- Pune-based BFSI legacy systems

Batch Modes and Duration

Duration: 8 weeks of structured curriculum plus 1 week of capstone (~2 months total)

Classroom

- Location: Archer Infotech, Kothrud, Pune
- Morning batch — 10:00 to 13:00
- Evening batch — 18:00 to 21:00

Live online

- Same hours as classroom batches
- Recordings available for review
- Zoom for live sessions
- MySQL Workbench / DBeaver
- GitHub for capstone
- Slack / WhatsApp for async Q&A

Weekend

- Saturday + Sunday, 09:00 to 13:00
- Stretches over ~3.5 months instead of 2.

Maximum 15 students per batch.

Fees

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession.

Indicative range: Rs. 20,000 – Rs. 90,000

- Single payment with early-bird discount
- EMI in 2 instalments at no extra cost
- Corporate sponsorship — invoiced with GST

Placement Support

Placement support starts from week 6. By the end of the curriculum your resume highlights real EXPLAIN-plan optimisation work, your GitHub has at least one schema-design and one replication / backup project, and you have completed at least two mock technical interviews.

We say placement support, not placement guarantee. Our support is unconditional, time-bound (six months after course completion), and includes free re-entry to a future batch's interview-prep sessions.

The placement process

- Week 6 — resume + LinkedIn rewrite
- Week 7 — GitHub portfolio cleanup
- Weeks 8–9 — two rounds of mock technical interviews
- Week 9 — HR mock and salary negotiation coaching
- Post-course — referrals via our 17-year alumni network
- Up to 6 months of continued support
- Free re-entry to future batch interview-prep sessions

Partner companies

- Persistent Systems
- TCS
- Infosys
- Cognizant
- Capgemini
- Wipro
- Atos / Eviden
- Mphasis
- Synechron

Prerequisites and How to Start

Prerequisites: basic computer use, willingness to commit 6–8 hours per week of practice. No prior SQL or programming experience required.

Suggested first steps

- Decide your mode — classroom, online live, or weekend
- Check the upcoming batch dates
- Book a free 30-minute counselling call
- Confirm enrolment and complete pre-course orientation (MySQL 8.4 install scripts)
- Show up to day one with a laptop running 64-bit OS

Frequently Asked Questions

How long does MySQL training in Pune take at Archer Infotech?

Approximately 2 months — 8 weeks plus 1 week of capstone. Weekend batch ~3.5 months.

Is MySQL still relevant in 2026?

Yes — Indeed Pune lists 600+ MySQL openings explicitly, plus ~1,000 backend / full-stack roles where MySQL appears alongside Postgres / Mongo. The LAMP / WordPress / digital-agency / e-commerce ecosystem runs heavily on MySQL.

MySQL or PostgreSQL?

MySQL for broadest entry-level / LAMP / WordPress hiring. Postgres for modern product engineering / fintech / analytics.

What is the salary of a MySQL Developer / DBA in Pune?

Indeed Pune Rs. 6.12 lakh average. Junior DBA Rs. 3.5–6 lakh per AmbitionBox. Mid-level Rs. 9–15 lakh. Senior DBA Rs. 15–25 lakh.

Will I work on real projects?

Yes — two capstone projects: (1) e-commerce schema with performance-tuned queries on 10M+ row dataset, (2) replicated production-style MySQL setup with backup.

Are weekend MySQL classes available in Pune?

Yes — Saturday and Sunday, 09:00–13:00, stretched over ~3.5 months.

What is the fee?

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession.

What support do I get after course completion?

Six months of placement support, referrals via our alumni network, mock interviews.

Are the named trainers actually teaching?

Ankita Hartale and Suraj Kudache personally lead every session.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.